

Testare automată E2E: Playwright vs. Cypress pe o platformă de e-commerce

Autori: Stan David Florin, Bejan Stefan, Elena Curecheriu, Andreea Onisie, Andrei
Stefan

Cuprins

1	Introducere	2
2	Scenariile de testare	4
3	Playwright	15
4	Cypress	22
5	Concluzii	30

Capitolul 1

Introducere

1.1 Contextul proiectului

Testarea automată a aplicațiilor web reprezintă o componentă esențială în ciclul de dezvoltare al oricărui sistem software modern. Pe măsură ce aplicațiile cresc în complexitate, testarea manuală devine insuficientă atât ca acoperire, cât și ca viteză de execuție.

Acest proiect are ca scop implementarea și compararea a două suite de teste automate E2E pentru aceeași aplicație țintă, folosind două framework-uri diferite: **Playwright** (Python) și **Cypress** (TypeScript). Prin implementarea duală a aceluiași scenariu de testare, proiectul oferă o perspectivă practică asupra diferențelor dintre cele două instrumente din punct de vedere al capabilităților tehnice și limitărilor fiecăruia.

1.2 Aplicația testată — NopCommerce

NopCommerce este o platformă de e-commerce open-source, construită pe ASP.NET Core, utilizată pe scară largă atât în mediul academic, cât și în producție. Oferă un flux complet de cumpărături: navigare prin catalog, filtrare produse, gestionarea coșului și listei de dorințe, înregistrare cont, autentificare, checkout în mai mulți pași și procesare plăți.

Această platformă reprezintă un candidat ideal pentru testare E2E, deoarece permite acoperirea unor scenarii realiste și complexe: interacțiuni asincrone (AJAX), formulare multi-pas, calcul dinamic de prețuri, gestionarea sesiunii și persistența datelor între sesiuni.

Testele din acest proiect au fost rulate local, pe o instanță NopCommerce găzduită prin Docker. Instanța publică oficială blochează sesiunile de browser automatizate prin protecția Cloudflare, care detectează traficul generat de framework-uri și returnează erori înainte ca testele să poată interacționa cu aplicația. Framework-ul Playwright a folosit în plus biblioteca **playwright-stealth** pentru a masca semnăturile de automatizare atunci

când instanța publică era accesată.

1.3 Obiectivele proiectului

Proiectul urmărește trei obiective principale:

1. **Definirea și documentarea scenariilor de testare** — 6 workflow-uri E2E complete, reprezentând user flow-urile reale pe o platformă de e-commerce.
2. **Implementarea scenariilor în Playwright și Cypress** — același set de scenarii este implementat în ambele framework-uri, permițând o comparație directă a abordărilor.
3. **Analiza comparativă** — evaluarea celor două instrumente din perspectiva configurării, scrierii testelor, depanării și a capabilităților tehnice avansate.

1.4 Configurarea mediului de testare

Aplicația NopCommerce rulează local prin Docker Compose, folosind fișierul `docker-compose.nopcommerce.yml`. Acesta orchestrează două servicii: **nopcommerce-db** (SQL Server 2022, portul 1433) și **nopcommerce-web** (aplicația NopCommerce, portul 8080), cu un *health check* care asigură că baza de date este funcțională înainte de pornirea aplicației. Pornirea mediului se face cu:

```
1 docker compose -f docker-compose.nopcommerce.yml up -d
```

Framework-ul Playwright este implementat în **Python**. Dependențele se instalează și browserele se descarcă cu:

```
1 pip install -r requirements.txt
2 playwright install
```

Testele se rulează cu:

```
1 pytest test_workflows.py
```

Framework-ul Cypress este implementat în **TypeScript** (modul strict). Dependențele se instalează cu:

```
1 npm install
```

Testele se rulează headless cu `npm test` sau în modul interactiv cu `npm run cy:open`.

Capitolul 2

Scenariile de testare

Suita de teste acoperă șase fluxuri end-to-end, alese astfel încât să reprezinte parcursurile critice ale unui utilizator real pe o platformă de e-commerce. Fiecare scenariu este descris prin precondiții, pași de testare și aserțiuni, însoțit de diagrama fluxului de execuție.

Notăție utilizată în diagrame:

- **Acțiune** — interacțiune directă cu interfața (click, completare formular, navigare);
- **Aserțiune** — verificare că sistemul se află în starea așteptată;
- **Soft assert** — verificare opțională; eșecul este înregistrat, dar testul continuă.

2.1 Workflow 1 — Cumpărături ca vizitator (Guest mega-flow)

Precondiții

- Sesiune de browser nouă (incognito), fără cont activ.
- Coșul de cumpărături și wishlist-ul sunt goale.

Pași de testare

1. Navigare prin meniu și aplicarea unui filtru asincron (16 GB RAM) pe categoria Notebooks.
2. **Assert:** răspunsul AJAX de filtrare returnează HTTP 200.
3. Adăugarea primului produs afișat în wishlist.
4. **Assert:** notificarea verde de confirmare este vizibilă.
5. Transferul produsului din wishlist în coș, actualizarea cantității la 2.

6. **Assert:** prețul total este exact dublul prețului unitar.
7. Checkout ca vizitator: completare adresă de facturare, selectare livrare *Next Day Air*, plată cu card.
8. **Assert:** URL-ul final conține `/checkout/completed`.
9. **Assert:** titlul paginii este *“Your order has been successfully processed!”*.

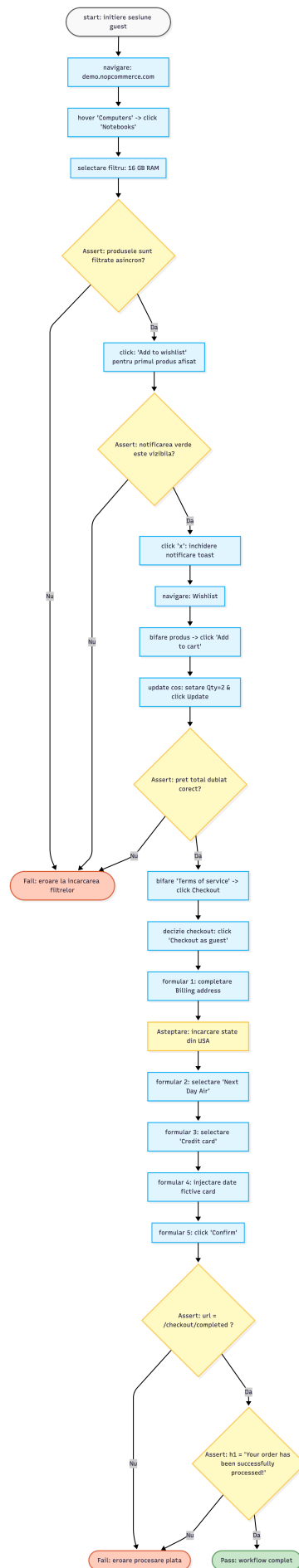


Figura 2.1: Diagrama fluxului ⁶ — Workflow 1: Guest mega-flow

2.2 Workflow 2 — Configurator PC cu prețuri dinamice

Precondiții

- Sesiune guest, coș gol.

Pași de testare

1. Schimbarea valutei la Euro.
2. **Assert:** simbolul `€` apare pe prețurile afișate.
3. Navigare la produsul *Build your own computer*; extragerea prețului de bază.
4. Selectare dinamică de componente (RAM, HDD, software) cu interceptare răspunsuri AJAX.
5. **Assert:** noul preț este mai mare decât prețul de bază.
6. Adăugare în coș; aplicarea cuponului invalid `cupon-fals-2026`.
7. **Soft assert:** mesajul de eroare roșu este vizibil; testul continuă indiferent.
8. Checkout cu adresă de livrare diferită de cea de facturare.
9. Selectare metodă de plată *Check/Money Order*.
10. **Assert:** mesaj de confirmare a comenzii.

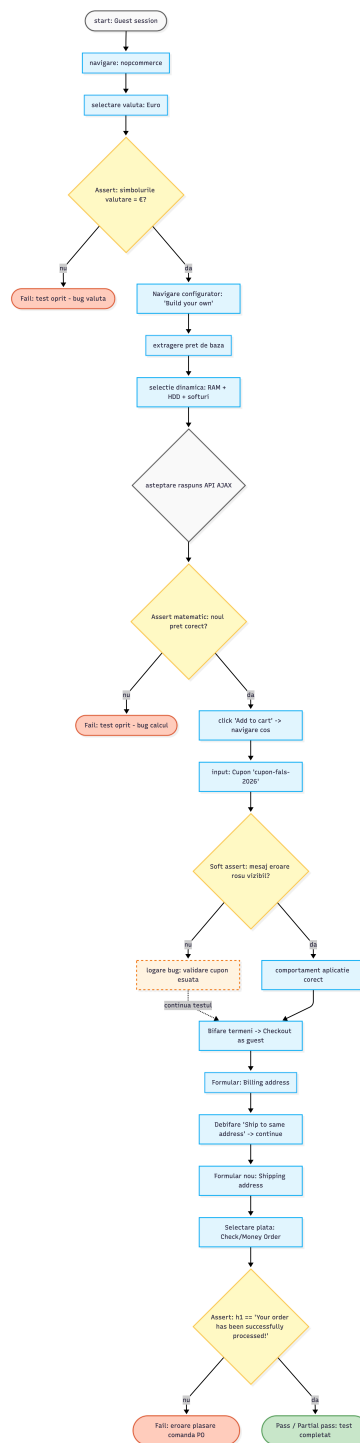


Figura 2.2: Diagrama fluxului — Workflow 2: Configurator PC

2.3 Workflow 3 — Tabelul de comparare produse

Precondiții

- Sesiune fără autentificare.
- Lista de comparare este goală (fără rulări anterioare nesalvate).

Pași de testare

1. Căutare *HTC* și adăugare la lista de comparare.
2. Căutare *Apple* și adăugare la lista de comparare.
3. Navigare la pagina de comparare via banner-ul verde de notificare.
4. **Assert:** tabelul HTML are exact 3 coloane.
5. **Assert:** rândul de nume conține ambele produse.
6. Click pe *Clear list*.
7. **Assert:** textul “*You have no items to compare.*” este afișat.

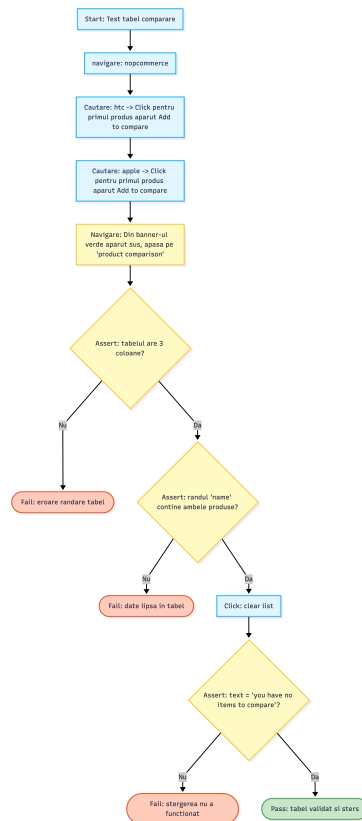


Figura 2.3: Diagrama fluxului — Workflow 3: Tabel comparare

2.4 Workflow 4 — Produs digital (livrare omisă)

Precondiții

- Sesiune nouă de browser, coș gol, fără produse fizice în așteptare.

Pași de testare

1. Navigare la secțiunea *Digital Downloads*.
2. Adăugare album digital în coș.
3. Checkout ca vizitator; completare adresă de facturare.
4. **Soft assert:** pasul de livrare fizică este omis automat, iar tab-ul de plată devine activ direct.
5. Selectare card ca metodă de plată; completare date fictive.
6. **Assert:** mesaj de confirmare a comenzii virtuale.

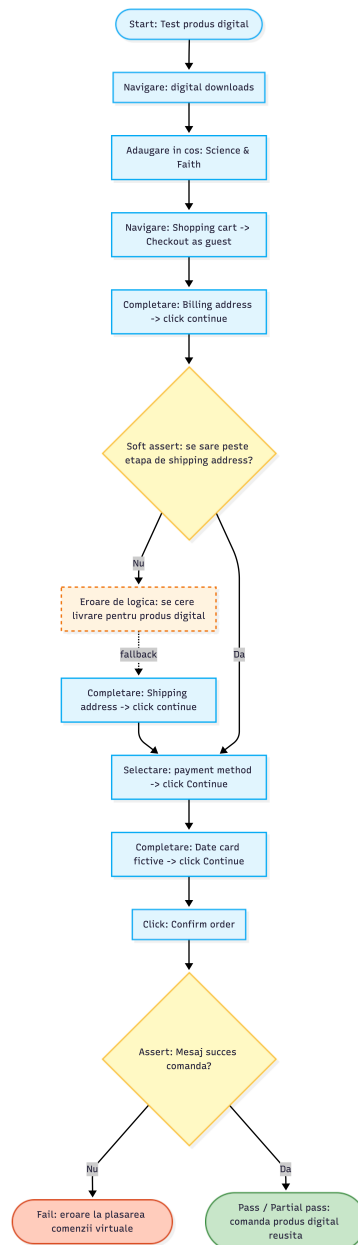


Figura 2.4: Diagrama fluxului — Workflow 4: Produs digital

2.5 Workflow 5 — Înregistrare cont și prima comandă

Precondiții

- Scriptul generează un email unic la fiecare rulare prin concatenarea unui timestamp (`testuser_{timestamp}@example.com`), evitând conflicte cu conturi existente.

Pași de testare

1. Navigare la formularul de înregistrare; completare date și email dinamic.

2. **Assert:** mesajul “*Your registration completed*” este afișat.
3. Căutare produs *Nokia Lumia* și adăugare în coș (utilizatorul este deja autentificat).
4. Inițiere checkout.
5. **Assert:** pasul de selecție *login/guest* este omis — contul tocmai creat este recunoscut automat.
6. Completare adresă, metodă de livrare, plată cu card.
7. **Assert:** URL conține */checkout/completed* și mesajul de succes este afișat.

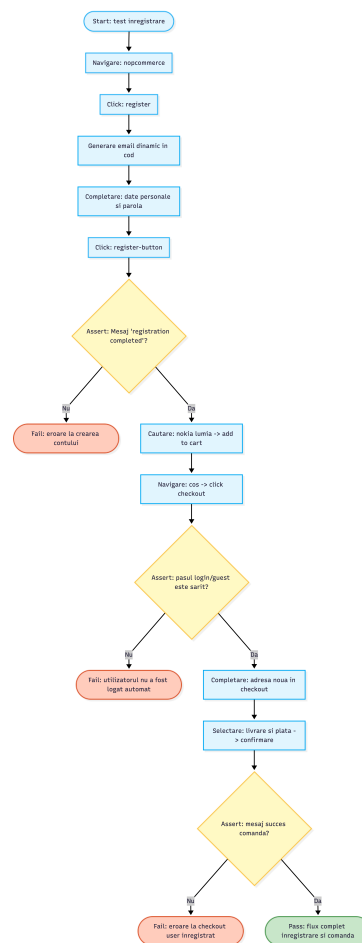


Figura 2.5: Diagrama fluxului — Workflow 5: Înregistrare și primă comandă

2.6 Workflow 6 — Persistența sesiunii (coș și adresă)

Precondiții

- Există un cont persistent cu credențiale hardcodate în script (`persistent.testers2026@example.com`).
- Dacă nu există, scriptul îl creează automat înainte de test.

Pași de testare

1. Autentificare cu contul persistent.
2. **Assert:** link-ul *My account* este vizibil.
3. Adăugare adresă nouă în agenda contului (cu un oraș generat dinamic).
4. Adăugare produs *Nokia Lumia* în coș.
5. Deconectare (*Log out*).
6. **Assert:** link-ul de login a reapărut în header.
7. Re-autentificare cu același cont.
8. **Assert:** badge-ul coșului indică cel puțin un produs — coșul a supraviețuit deconectării.
9. Navigare la checkout.
10. **Assert:** adresa salvată anterior apare în dropdown-ul de facturare.

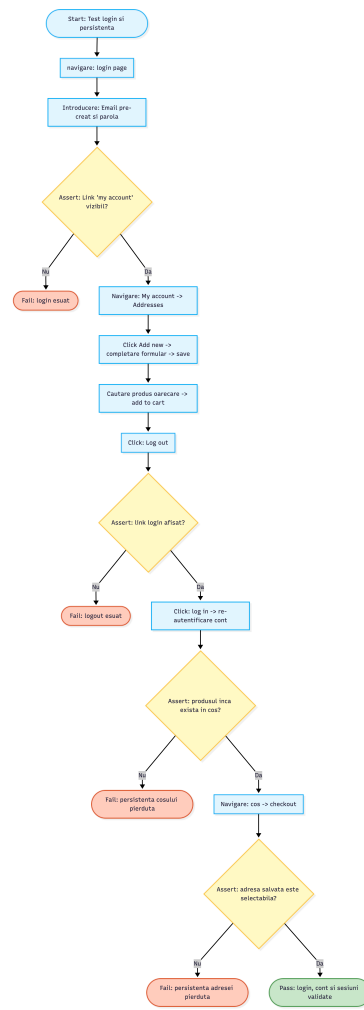


Figura 2.6: Diagrama fluxului — Workflow 6: Persistența sesiunii

Capitolul 3

Playwright

3.1 Ce este Playwright?

Playwright este un framework open-source de automatizare a browserelor, dezvoltat de Microsoft și lansat în 2020. Este conceput pentru a testa aplicații web moderne, oferind suport nativ pentru trei motoare de browser: **Chromium**, **Firefox** și **WebKit** (Safari). Spre deosebire de predecesori precum Selenium, Playwright controlează browserul printr-un protocol binar propriu (DevTools Protocol), ceea ce îi conferă viteză și fiabilitate superioare.

Playwright pune la dispoziție API-uri pentru Python, JavaScript/TypeScript, Java și C#. În cadrul acestui proiect a fost utilizat **API-ul sincron Python**, integrat cu runner-ul de teste `pytest`.

3.2 Facilități principale

3.2.1 Suport multi-browser

Playwright rulează aceleași teste pe Chromium, Firefox și WebKit fără modificări de cod. Fiecare motor este distribuit împreună cu biblioteca, eliminând dependența de versiunile instalate local pe mașina de test.

3.2.2 Auto-wait

Playwright implementează un mecanism de așteptare automată (*auto-wait*): înainte de a interacționa cu un element, framework-ul verifică automat că acesta este vizibil, activat și stabil în DOM. Acest comportament elimină necesitatea apelurilor explicite de tip `sleep()` sau `wait_for_element()` în marea majoritate a cazurilor.

3.2.3 Generare automată de teste — Codegen

Playwright include un instrument de înregistrare a interacțiunilor numit `codegen`. La rularea comenzii, se deschide un browser în care utilizatorul interacționează normal cu aplicația, iar Playwright generează automat codul de test corespunzător în timp real.

```
1 playwright codegen http://localhost:8080
```

Codul generat poate fi copiat direct în fișierul de test și rafinat ulterior. Aceasta accelerează semnificativ scrierea testelor pentru fluxuri noi, eliminând nevoia de a identifica manual selectorii elementelor.

3.2.4 Interceptarea rețelei

API-ul `page.expect_response()` permite atașarea unui context de așteptare la un apel de rețea specific, identificat prin URL sau expresie regulată. Testul este blocat până când răspunsul este recepționat, moment în care se poate inspecta statusul sau corpul răspunsului. Aceasta este o alternativă robustă la așteptările bazate pe timp fix.

3.2.5 Locatori rezilienți

Playwright introduce conceptul de *Locator*, o referință lazy către un element din DOM. Spre deosebire de un `ElementHandle` clasic (capturat la un moment de timp), un `Locator` re-caută elementul la fiecare interacțiune, adaptându-se la re-randările dinamice ale paginii.

3.2.6 Izolarea contextelor de browser

Fiecare test poate rula într-un context de browser separat (`BrowserContext`), echivalent cu un profil de utilizator distinct. Contextele sunt izolate: cookie-uri, `localStorage` și cache-ul nu se propagă între ele. Plugin-ul `pytest-playwright` creează automat câte un context și o pagină pentru fiecare funcție de test.

3.2.7 Mod Trace și raportare

Playwright include un instrument de înregistrare a execuției (*Trace Viewer*) care captează screenshot-uri, snapshot-uri DOM, apeluri de rețea și timeline-ul acțiunilor. Fișierul de trace poate fi inspectat ulterior în browser, fără a rula din nou testul.

3.3 Configurarea proiectului

3.3.1 Dependențe

Dependențele proiectului sunt definite în `requirements.txt`:

Listing 3.1: requirements.txt

```
1 playwright==1.58.0
2 pytest==9.0.3
3 pytest-playwright==0.7.2
4 pytest-base-url==2.1.0
5 playwright-stealth==2.0.3
```

Instalarea se face cu:

```
1 pip install -r requirements.txt
2 playwright install
```

Comanda `playwright install` descarcă binarele pentru Chromium, Firefox și WebKit.

3.3.2 Bypass Cloudflare cu playwright-stealth

Instanța publică `demo.nopcommerce.com` este protejată de Cloudflare, care detectează și blochează sesiunile de browser automatizate prin analiza fingerprint-ului JavaScript (prezența variabilei `navigator.webdriver`, absența plugin-urilor, timingul evenimentelor).

Pentru a ocoli această protecție, proiectul utilizează biblioteca `playwright-stealth`, care modifică proprietățile expuse de browser astfel încât sesiunea automată să fie indistinctibilă de una umană. Integrarea se face printr-un fixture `pytest` cu `autouse=True`, aplicat global tuturor testelor:

Listing 3.2: Fixture global pentru stealth mode

```
1 @pytest.fixture(autouse=True)
2 def apply_stealth(page: Page):
3     Stealth().apply_stealth_sync(page)
```

Prin declararea `autouse=True`, fixture-ul este injectat automat în fiecare test fără a fi necesar să fie menționat explicit în semnătura funcției.

3.4 Implementarea testelor

3.4.1 Structura unui test

Fiecare test este o funcție Python prefixată cu `test_`, care primește ca argument obiectul `page` furnizat de plugin-ul `pytest-playwright`. Acesta reprezintă o instanță de pagină browser gata de utilizare, izolată față de celelalte teste.

Listing 3.3: Structura de bază a unui test Playwright

```
1 def test_guest_shopping(page: Page):
2     page.goto(BASE_URL)
3     page.locator("a.menu_link[href='/computers']").hover()
4     page.locator("a[href='/notebooks']").click()
5     # ...
6     expect(page.locator(".order-completed .title")).to_have_text(
7         "Your order has been successfully processed!"
8     )
```

3.4.2 Interceptarea cererilor AJAX

Un pattern central în suita de teste este așteptarea răspunsurilor AJAX înainte de a continua execuția. Fără această sincronizare, testul ar interacționa cu elemente care nu reflectă încă rezultatul operației asincrone.

Exemplul următor verifică că filtrarea produselor după RAM s-a încheiat cu succes înainte de a continua:

Listing 3.4: Interceptare răspuns AJAX la aplicarea filtrului

```
1 with page.expect_response(
2     lambda response: "demo.nopcommerce.com" in response.url
3     and response.request.resource_type in ["xhr", "fetch"]
4 ) as response_info:
5     page.locator("input[id='attribute-option-9']").check()
6
7 assert response_info.value.status == 200, \
8     "Filtrarea asincron (AJAX) nu a returnat status 200"
```

Blocul `with page.expect_response(...)` funcționează ca un context manager: acțiunea din interior declanșează cererea, iar Playwright așteaptă răspunsul înainte de a ieși din bloc.

3.4.3 Verificarea prețurilor dinamice

Workflow-ul 2 (Configuratorul PC) implică selectarea unor componente care modifică prețul afișat prin AJAX. Testul extrage prețul înainte și după selecție și verifică matematic

că actualizarea a avut loc:

Listing 3.5: Verificare calcul preț dinamic

```
1 base_price_text = page.locator("div.product-price span").inner_text()
2 base_price = float(re.sub(r'^\d.', '', base_price_text))
3
4 with page.expect_response(re.compile(r".*productdetails_attributechange
5     .*",
6                               re.IGNORECASE)):
7     page.locator("input[name='product_attribute_3'][value='7']").check()
8
9 new_price_text = page.locator("div.product-price span").inner_text()
10 new_price = float(re.sub(r'^\d.', '', new_price_text))
11
12 assert new_price > base_price, \
13     f"Pretul nu s-a actualizat corect! {new_price} vs {base_price}"
```

3.4.4 Soft assertions

Un *soft assert* este o verificare al cărei eșec nu oprește execuția testului. Playwright nu oferă un mecanism nativ pentru soft assertions, însă acestea se implementează simplu prin capturarea excepției:

Listing 3.6: Implementare soft assert pentru cuponul invalid

```
1 try:
2     page.wait_for_selector("div.message-failure", timeout=5000)
3     expect(page.locator("div.message-failure").first).to_have_text(
4         re.compile(r"coupon", re.IGNORECASE)
5     )
6 except (AssertionError, Exception) as e:
7     print(f"\nMesajul de eroare pentru cupon nu a aparut. "
8         f"Soft-assert ignorat. Detalii: {e}")
9     pass
```

Dacă mesajul de eroare nu apare (comportament întâlnit uneori pe instanța demo), testul înregistrează situația în consolă și continuă spre pașii următori.

3.4.5 Gestionarea sesiunilor persistente

Workflow-ul 6 testează că datele (coș, adrese) supraviețuiesc unui ciclu de logout/login. Pentru aceasta, testul utilizează un cont cu credențiale fixe, creat o singură dată:

Listing 3.7: Funcție helper pentru autentificare

```
1 PERSISTENT_EMAIL = "persistent.testers2026@example.com"
2 PERSISTENT_PASSWORD = "PersistTest123!"
```

```

3
4 def _login(page: Page):
5     page.goto(f"{BASE_URL}/login")
6     page.locator("#Email").fill(PERSISTENT_EMAIL)
7     page.locator("#Password").fill(PERSISTENT_PASSWORD)
8     page.locator("button.login-button").click()
9     page.wait_for_load_state("networkidle")

```

Funcția `_ensure_account_exists()` încearcă înregistrarea contului; dacă primește o eroare (contul există deja), continuă direct cu autentificarea — eliminând necesitatea unui setup manual înainte de prima rulare.

3.4.6 Adresă de email unică per rulare

Workflow-ul 5 (înregistrare cont) necesită un email unic la fiecare rulare pentru a evita eroarea *“account already exists”*. Soluția este generarea unui timestamp:

Listing 3.8: Generare email unic cu timestamp

```

1 unique_email = f"testuser_{int(time.time())}@example.com"

```

3.5 Avantaje

- **Suport multi-browser nativ:** Chromium, Firefox și WebKit testate cu același cod, fără configurare suplimentară.
- **Interceptare rețea puternică:** `expect_response()` și `route()` permit sincronizare și modificare a cererilor HTTP direct din test.
- **Auto-wait robust:** elimină majoritatea așteptărilor explicite, reducând fragilitatea testelor la variații de latență.
- **Izolare contexte:** fiecare test pornește cu un browser curat, fără interferențe între sesiuni.
- **Ecosistem Python:** integrare naturală cu `pytest`, `fixtures`, parametrizare, și librăriile de analiză/date din ecosistemul Python.
- **Bypass Cloudflare:** prin `playwright-stealth`, testele pot rula pe instanțe protejate fără a necesita un mediu local.
- **Trace Viewer:** debugging post-mortem fără re-rularea testelor.

3.6 Dezavantaje

- **Configurare inițială mai complexă:** necesită instalarea separată a binarelor de browser și configurarea mediului Python, față de un simplu `npm install`.
- **Verbozitate:** codul Python tinde să fie mai explicit față de echivalentul Cypress, mai ales în cazul selectoarelor și așteptărilor condiționale.
- **Fără Time Travel nativ:** Playwright nu oferă un echivalent al funcției de *time travel debugging* din Cypress App — reluarea pas cu pas a execuției necesită activarea Trace Viewer înainte de rulare.
- **Soft assertions manuale:** absența unui mecanism nativ pentru soft assertions obligă la construcții `try/except` repetitive.
- **Dependență de stealth pentru site-uri protejate:** pe instanțe publice cu protecție anti-bot, este necesară o bibliotecă externă care poate deveni incompatibilă cu versiuni noi de Playwright.

Capitolul 4

Cypress

4.1 Ce este Cypress?

Cypress este un framework de testare end-to-end conceput exclusiv pentru aplicații web, lansat în 2017 de Cypress.io. Spre deosebire de Playwright, care controlează browserul din exterior printr-un protocol de automatizare, Cypress rulează **direct în interiorul browserului**, în același proces JavaScript cu aplicația testată. Această arhitectură îi conferă acces nativ la DOM, la rețea și la starea internă a aplicației, fără latența unui strat intermediar.

Cypress suportă **Chromium** (Chrome, Edge) și **Firefox**. Suportul pentru WebKit/-Safari este disponibil în versiuni experimentale. Limbajul de scripting este **JavaScript** sau **TypeScript**, iar în cadrul acestui proiect a fost utilizat **TypeScript în mod strict**.

4.2 Facilități principale

4.2.1 Time Travel Debugging

Cypress App (interfața grafică interactivă) înregistrează un snapshot al DOM-ului înainte și după fiecare comandă executată. Trecând cu mouse-ul peste oricare pas din log, interfața restaurează vizual starea aplicației la acel moment, permițând inspecția elementelor exact în momentul în care comanda a acționat asupra lor. Aceasta este una din facilitățile cele mai apreciate de Cypress, eliminând nevoia de a re-rula testul pentru a diagnostica un eșec.

4.2.2 Retry-ability automat

Comenzile Cypress nu returnează imediat o valoare — ele sunt *lazy* și se re-evaluează automat până când condiția este îndeplinită sau timeout-ul expiră. De exemplu, `cy.get('.element').should(...)` va re-interoga DOM-ul repetat timp de până la 10 secunde (implicit) înainte de a eșua.

Această caracteristică elimină necesitatea așteptărilor explicite în marea majoritate a cazurilor.

4.2.3 Interceptarea rețelei cu `cy.intercept()`

Cypress oferă un API de interceptare a cererilor HTTP direct la nivelul browserului. Comanda `cy.intercept()` permite:

- atribuirea unui alias unei cereri și așteptarea ei cu `cy.wait('@alias')`;
- inspecția răspunsului (status, body);
- modificarea sau blocarea cererilor pentru simularea erorilor.

4.2.4 Comenzi reutilizabile și helper functions

TypeScript permite definirea de funcții helper tipizate, utilizabile în orice test. Proiectul folosește această facilitare extensiv pentru a evita duplicarea codului de checkout, autentificare și completare formulare.

4.2.5 Plugin ecosystem

Cypress dispune de un ecosistem de plugin-uri pentru extinderea capabilităților native. Proiectul utilizează `cypress-real-events`, care simulează evenimente native de browser (hover, drag, scroll real) — necesare pentru meniurile dropdown care nu răspund la evenimentele sintetice Cypress standard.

4.2.6 Cypress App — interfața interactivă

Comanda `npm run cy:open` deschide Cypress App, o interfață grafică în care testele pot fi rulate individual, cu vizualizarea în timp real a fiecărui pas, a snapshot-urilor DOM și a log-ului de rețea. Aceasta accelerează semnificativ procesul de scriere și depanare a testelor.

4.3 Configurarea proiectului

4.3.1 Dependente

Dependențele sunt definite în `package.json`:

Listing 4.1: Instalare dependențe Cypress

```
1 npm install
```


Pachetele principale:

- `cypress@15.14.0` — framework-ul principal;
- `typescript@6.0.3` — compilatorul TypeScript;
- `cypress-real-events@1.15.0` — simulare evenimente native de browser.

4.3.2 Fișierul de configurare

Configurația relevantă din `cypress.config.js`:

Listing 4.2: `cypress.config.js`

```
1 module.exports = defineConfig({
2   chromeWebSecurity: false,
3   video: false,
4   screenshotOnRunFailure: true,
5   e2e: {
6     baseUrl: 'http://localhost:8080',
7     viewportWidth: 1440,
8     viewportHeight: 900,
9     defaultCommandTimeout: 10000,
10    specPattern: 'cypress/e2e/**/*.cy.ts',
11  },
12 });
```

`chromeWebSecurity: false` este necesar pentru a permite accesul cross-origin în fluxurile de checkout, unde unele resurse sunt încărcate de pe domenii diferite. `screenshotOnRunFailure: true` salvează automat un screenshot la fiecare test eșuat, util pentru diagnosticarea problemelor în rulările headless din CI.

4.3.3 Scripturi disponibile

- `npm test` — rulare headless completă (`cypress run`);
- `npm run cy:open` — Cypress App, mod interactiv;
- `npm run cy:run:chrome` — rulare headed în Chrome;
- `npm run cy:open:local` — mod interactiv cu `baseUrl` explicit pe Docker.

4.4 Implementarea testelor

4.4.1 Structura testelor

Testele sunt organizate cu `describe` pentru grupare și `it` pentru cazuri individuale, urmând convenția standard Mocha (utilizată intern de Cypress):

Listing 4.3: Structura de organizare a testelor

```

1 describe('NopCommerce Workflows', () => {
2   describe('Guest Purchase Journeys', () => {
3     it('Guest shopping flow: wishlist -> cart -> checkout success', ()
4       => {
5       cy.visit('/');
6       // ...
7     });
8   });
9
10  describe('Account-Based Journeys', () => {
11    it('Register and place an order in the same session', () => {
12      // ...
13    });
14  });
15 });

```

4.4.2 Funcții helper reutilizabile

O parte semnificativă a codului este extrasă în funcții helper definite la nivelul fișierului, eliminând duplicarea între teste:

Listing 4.4: Helper pentru completarea formularelor de adresă

```

1 type AddressPayload = {
2   firstName: string; lastName: string; email: string;
3   city: string; address1: string;
4   zipPostalCode: string; phoneNumber: string;
5 };
6
7 const fillAddressForm = (prefix: string, payload: AddressPayload) => {
8   cy.intercept('GET', '**/getstatesbycountryid*').as('getStates_${prefix}');
9   cy.get(`#${prefix}_CountryId`).select('237');
10  cy.get(`#${prefix}_StateProvinceId option`)
11    .should('have.length.greaterThan', 1);
12  cy.get(`#${prefix}_StateProvinceId`).select(1);
13  cy.get(`#${prefix}_FirstName`).clear().type(payload.firstName);
14  // ...
15 };

```

Funcția `fillAddressForm` este apelată cu un prefix diferit pentru fiecare tip de adresă (`BillingNewAddress`, `ShippingNewAddress`, `Address`), construind dinamic ID-urile câmpurilor.

4.4.3 Interceptarea cererilor AJAX

Interceptarea cererilor AJAX în Cypress se face prin `cy.intercept()` combinat cu `cy.wait()`. Exemplul următor verifică că răspunsul la filtrarea produselor returnează HTTP 200:

Listing 4.5: Interceptare AJAX la aplicarea filtrului

```
1 cy.intercept('**/*').as('ajaxFilter');
2 cy.get("input[id='attribute-option-9']").check({ force: true });
3 cy.wait('@ajaxFilter').its('response.statusCode').should('eq', 200);
```

Pentru cereri mai specifice, se folosește o expresie regulată:

Listing 4.6: Interceptare cerere de actualizare atribut produs

```
1 cy.intercept(/productdetails_attributechange/i).as('attrChange1');
2 cy.get("input[name='product_attribute_3'][value='7']").check({ force:
   true });
3 cy.wait('@attrChange1');
```

4.4.4 Verificarea prețurilor dinamice

Funcția utilitară `toNumber()` curăță textul afișat și îl convertește la număr, permițând comparații matematice:

Listing 4.7: Extragere și verificare preț dinamic

```
1 const toNumber = (text: string) =>
2   Number(text.replace(/[^\d.]/g, '').replace(/,/g, ''));
3
4 cy.get('div.product-price span').invoke('text').then((basePriceText) =>
5   {
6     const basePrice = toNumber(basePriceText);
7
8     // selectie componente + asteptare AJAX
9     cy.intercept(/productdetails_attributechange/i).as('attrChange');
10    cy.get("input[name='product_attribute_3'][value='7']").check({ force:
11      true });
12    cy.wait('@attrChange');
13
14    cy.get('div.product-price span').invoke('text').then((newPriceText) =>
15      {
16        const newPrice = toNumber(newPriceText);
17        expect(newPrice).to.be.greaterThan(basePrice);
18      });
19  });
```

4.4.5 Soft assertions

Cypress nu oferă soft assertions nativ. Acestea se implementează prin `cy.get('body').then()` care permite inspecția condiționată a DOM-ului fără a opri execuția testului:

Listing 4.8: Soft assert pentru cuponul invalid

```
1 cy.get('body').then(($body) => {
2   if ($body.find('div.message-failure').length > 0) {
3     cy.get('div.message-failure').first().invoke('text').then((
4       failureText) => {
5       const normalized = failureText.trim().toLowerCase();
6       if (normalized.length > 0) {
7         expect(normalized).to.contain('coupon');
8       } else {
9         cy.log('Coupon container present but empty, skipping assertion
10           .');
11       }
12     });
13   } else {
14     cy.log('Coupon error banner did not appear, continuing as soft
15       assertion.');
```

4.4.6 Hover cu cypress-real-events

Meniurile dropdown ale NopCommerce se activează la hover. Evenimentele sintetice Cypress (`cy.trigger('mouseover')`) nu declanșează CSS `:hover` în toate cazurile. Plugin-ul `cypress-real-events` rezolvă aceasta prin simularea unui eveniment nativ:

Listing 4.9: Hover nativ pentru meniul dropdown

```
1 cy.get("a.menu__link[href='/computers']").realHover();
2 cy.get("a[href='/notebooks']").click({ force: true });
```

4.4.7 Gestionarea metodelor de plată variabile

Pe instanța demo, metodele de plată disponibile pot varia. Funcția helper `selectPaymentMethodAndContinue` gestionează acest caz prin inspecția DOM-ului înainte de selecție:

Listing 4.10: Selecție robustă a metodei de plată

```
1 const selectPaymentMethodAndContinue = () => {
2   cy.get('body').then(($body) => {
3     const creditCardMethod = $body.find(
4       "input[type='radio'][name='paymentmethod'][value*='Manual']"
5     );
```

```

6   const allMethods = $body.find(
7     "input[type='radio'][name='paymentmethod']"
8   );
9
10  if (creditCardMethod.length > 0) {
11    cy.wrap(creditCardMethod.first()).check({ force: true });
12    return;
13  }
14  if (allMethods.length > 0) {
15    cy.wrap(allMethods.first()).check({ force: true });
16  }
17  });
18
19  cy.get('.payment-method-next-step-button:visible').click();
20 };

```

4.5 Avantaje

- **Developer experience superior:** Time Travel Debugging, hot reload în Cypress App și feedback vizual în timp real accelerează dramatic procesul de scriere a testelor.
- **Retry-ability built-in:** eliminarea așteptărilor explicite face testele mai concise și mai reziliente la variații de latență.
- **cy.intercept() intuitiv:** API-ul de interceptare este simplu și expresiv, cu suport pentru aliasuri și aserțiuni înlănțuite.
- **TypeScript nativ:** tipizarea strictă detectează erori de selector sau de tip la momentul compilării, nu la runtime.
- **Setup rapid:** `npm install + npx cypress open` este suficient pentru a începe — fără instalare separată de binarele de browser.
- **Screenshots automate la eșec:** documentarea eșecurilor fără configurare suplimentară.

4.6 Dezavantaje

- **Suport browser limitat:** Cypress suportă nativ doar Chromium și Firefox. Suportul WebKit/Safari este experimental și nu acoperă toate cazurile de utilizare.
- **JavaScript/TypeScript exclusiv:** nu există suport pentru Python, Java sau C#, limitând adoptarea în echipe cu alt stack tehnic.

- **Fără stealth mode:** Cypress nu are un echivalent al **playwright-stealth**. Testele pe instanțe protejate de Cloudflare necesită un mediu local (Docker), ceea ce crește complexitatea infrastructurii.
- **Arhitectură single-tab:** Cypress rulează într-o singură filă de browser; scenariile care implică mai multe tab-uri sau ferestre sunt dificil de testat.
- **Soft assertions manuale:** similar cu Playwright, absența unui mecanism nativ obligă la construcții `cy.get('body').then()` verbale.
- **Asincronicitate implicită:** comenzile Cypress sunt înlanțuite asincron (*command queue*), iar accesul la valori în afara callback-urilor `.then()` este o sursă frecventă de confuzie pentru utilizatorii noi.

Capitolul 5

Concluzii

5.1 Playwright

Playwright s-a dovedit un instrument robust și flexibil pentru testarea aplicației Nop-Commerce pe instanța publică, protejată de Cloudflare. Capacitatea de a intercepta și valida răspunsuri de rețea prin `page.expect_response()` a permis sincronizarea precisă cu operațiunile AJAX ale platformei, eliminând necesitatea așteptărilor bazate pe timp fix. Mecanismul de auto-wait a redus fragilitatea testelor în fața variațiilor de latență ale rețelei.

Un avantaj semnificativ a fost integrarea cu ecosistemul Python: fixtures-urile `pytest`, tipizarea prin type hints și biblioteca `playwright-stealth` au permis construirea unei suite funcționale pe un site cu protecție anti-bot, fără a recurge la un mediu local. Izolarea contextuală automată între teste a asigurat că fiecare rulare pornește dintr-o stare curată, fără interferențe de sesiune.

Principala limitare întâmpinată a fost verbozitatea codului în cazul scenariilor condiționale — absența soft assertions native a obligat la construcții `try/except` repetitive. De asemenea, lipsa unui echivalent vizual al Cypress App a făcut ca depănarea eșecurilor să fie mai laborioasă, necesitând activarea Trace Viewer înainte de rulare.

În ansamblu, Playwright reprezintă o alegere solidă pentru suite de testare complexe, multi-browser, în echipe cu experiență Python sau în scenarii în care controlul granular al rețelei este esențial.

5.2 Cypress

Cypress a oferit cel mai fluid proces de scriere și depănare a testelor din cadrul acestui proiect. Funcționalitatea Time Travel Debugging, combinată cu retry-ability-ul automat, a redus semnificativ timpul necesar identificării și corectării eșecurilor. API-ul `cy.intercept()` s-a dovedit intuitiv și expresiv, permițând atât sincronizarea cu AJAX-ul

platformei, cât și inspecția răspunsurilor HTTP cu sintaxă minimală.

Decizia de a rula testele pe o instanță Docker locală a eliminat problema Cloudflare, oferind un mediu stabil și reproductibil. TypeScript strict a contribuit la calitatea codului, detectând erori de selector și de tip la compilare, nu la runtime. Funcțiile helper tipizate (`fillAddressForm`, `selectPaymentMethodAndContinue`) au redus duplicarea codului și au crescut lizibilitatea suitei.

Limitările principale au fost legate de arhitectura framework-ului: suportul exclusiv pentru Chromium și Firefox, absența unui mecanism de stealth și restricția la un singur tab de browser au impus compromisuri de infrastructură (Docker) și au limitat acoperirea cross-browser. Asincronicitatea implicită a *command queue*-ului Cypress a reprezentat o curbă de învățare pentru scenariile care necesitau accesul la valori intermediare.

Cypress rămâne alegerea optimă pentru proiecte în care developer experience-ul și viteza de iterație au prioritate, în special când testele rulează pe o infrastructură controlată și stack-ul echipei este JavaScript/TypeScript.

5.3 De ce se preferă Playwright în industrie

În ultimii ani, Playwright a câștigat o poziție dominantă în industrie, depășind Cypress ca framework de referință pentru testarea E2E. Raportul *State of JS 2023* și sondajele comunității indică o adoptare în creștere accelerată, susținută de mai mulți factori structurali.

Suport multi-browser real

Playwright testează nativ pe Chromium, Firefox și WebKit cu același cod și aceeași comandă de rulare. Aceasta este o cerință frecventă în organizațiile care livrează produse pe multiple platforme și browsere. Cypress acoperă doar Chromium și Firefox, iar suportul WebKit rămâne experimental — o limitare semnificativă pentru echipele care trebuie să garanteze compatibilitatea Safari.

Suport multi-limbaj

Playwright oferă API-uri oficiale pentru Python, TypeScript, Java și C#. Această flexibilitate îl face accesibil echipelor cu stack-uri tehnice diverse și permite integrarea testelor E2E în proiecte care nu sunt construite pe JavaScript. Cypress este limitat la JavaScript/TypeScript, ceea ce restrânge adoptarea în organizații cu echipe mixte.

Arhitectură fără limitări de tab

Playwright suportă nativ scenarii cu mai multe pagini și tab-uri de browser, ferestre noi și contexte de browser independente. Cypress, prin arhitectura sa in-browser, nu poate gestiona mai multe tab-uri simultan — o limitare care devine vizibilă în aplicații moderne cu fluxuri OAuth, plăți cu redirect extern sau notificări în ferestre pop-up.

Viteza de execuție și paralelism

Playwright rulează testele în paralel pe mai mulți workeri în mod implicit, fără configurare suplimentară. Izolarea prin `BrowserContext` este mai ușoară și mai rapidă decât pornirea unui proces Cypress complet pentru fiecare test. În suite mari, diferența de timp de execuție devine semnificativă.

Integrare în CI/CD

Playwright este proiectat explicit pentru medii headless și CI/CD: binarele de browser sunt descărcate și gestionate automat, nu există dependențe de un display server (Xvfb), iar rapoartele HTML și fișierele de trace sunt generate nativ. Cypress necesită configurare suplimentară în CI pentru medii headless și are limitări legate de licențiere pentru Cypress Cloud (dashboard-ul de raportare).

Tabel comparativ

Criteriu	Playwright	Cypress
Browsere suportate	Chromium, Firefox, WebKit	Chromium, Firefox
Limbaje de scripting	Python, TS, Java, C#	JavaScript / TypeScript
Multi-tab / multi-window	Da	Nu (limitare arhitecturală)
Stealth / bypass bot	Da (plugin oficial)	Nu
Paralelism nativ	Da	Parțial (Cypress Cloud)
Time Travel Debugging	Trace Viewer (post-mortem)	Da (live, în Cypress App)
Developer experience	Bun	Excelent
Setup inițial	Mediu	Simplu
Integrare CI/CD	Excelentă	Bună

Tabela 5.1: Comparație Playwright vs. Cypress

Concluzie generală

Alegerea între Playwright și Cypress depinde de contextul specific al proiectului. Cypress rămâne superior din punct de vedere al experienței de dezvoltare și al vitezei de iterație

în proiecte mici și medii cu echipe JavaScript. Playwright se impune în organizații care necesită acoperire cross-browser reală, flexibilitate de limbaj, scenarii complexe cu mai multe pagini sau integrare nativă în pipeline-uri CI/CD la scară mare.

Tendința din industrie favorizează Playwright tocmai pentru că aceste cerințe — multi-browser, multi-limbaj, CI-first — sunt din ce în ce mai frecvente în proiectele moderne, unde calitatea software-ului trebuie garantată pe platforme și configurații diverse, nu doar pe Chrome.